

# SQL Optimization: Next Steps

O'Reilly Online Live Training



**I'm Haki Benita**

A software developer and a technical lead.

I'm interested in databases, web development, software design and performance tuning.

**[hakibenita.com](https://hakibenita.com)**



# About Me

- Started as an Oracle DBA
- Worked on DWH and DSS systems
- Leading a team of developers and DBAs
- Got into web development, mainly Python and Django
- Leading the development of a large ticketing platform



# POLL

**What is your job title?**

- DBA
- Developer
- Sysadmin / SRE
- Analyst / Data ops / Data scientist
- Student
- Management
- Other



# POLL

## What is your main use of data / SQL?

- Ad-Hoc Reports (sales reports, operational reports, etc.)
- Research / Analysis (BI / DWH / Dashboards/ Data exploration etc.)
- Development (Backend Development / ETL / Data pipelines etc.)



# POLL

**How would you rate your level of proficiency with databases?**

- Novice
- Intermediate
- Advanced



# POLL

**What is your primary database engine?**

- PostgreSQL
- MySQL
- SQL Server
- Oracle
- SQLite
- Other?



# What You'll Learn

- How to avoid common mistakes in SQL
- How to evaluate and optimize SQL performance





# SQL History

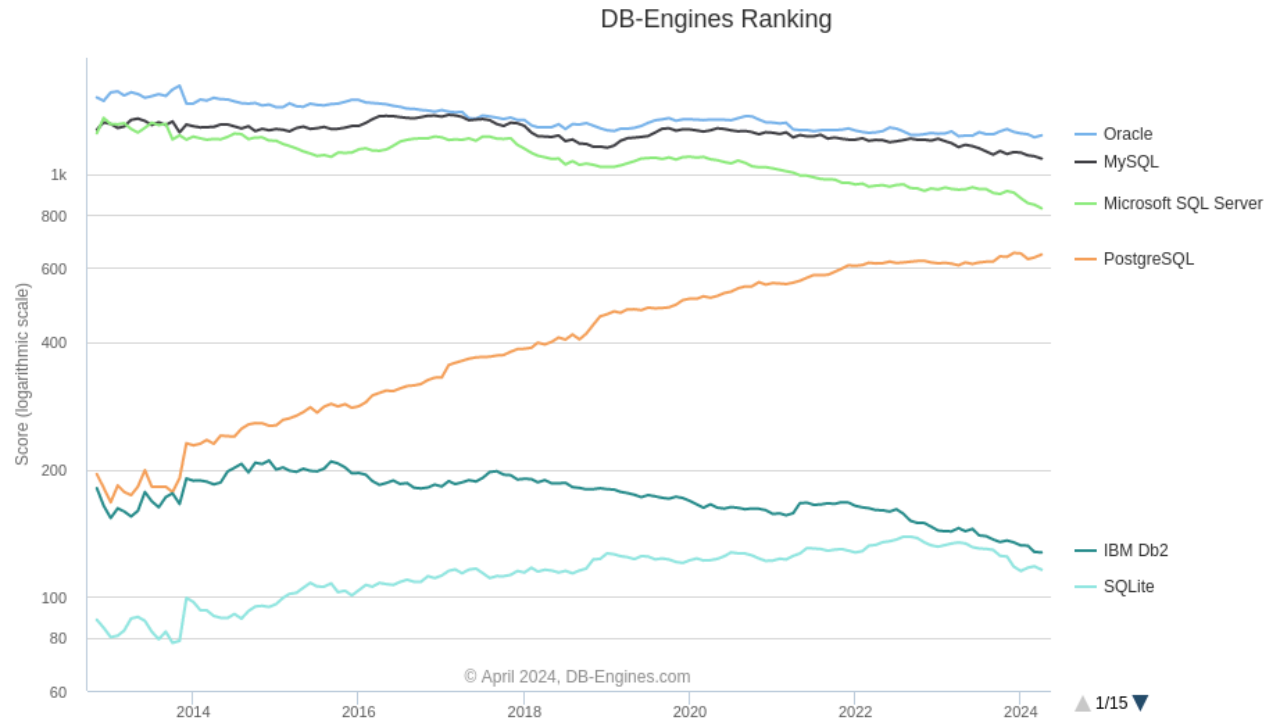
- **Structured Query Language**
- Used for interacting with relational databases (RDBMS)
- Invented in the early 70s at IBM
- Based on work by Edgar F. Codd
- Became a standard in 1986 (ANSI-86)
- Standard revised in 1992 (ANSI-92)

 [Important Papers: Codd and the Relational Model](#)

 [Comparing Database Types: How Database Types Evolved to Meet Different Needs](#)

# PostgreSQL

- Based on the Berkeley POSTGRES project from 1986
- Originally named Postgres95
- Free Open Source
- Growing fast!





# Anatomy of an SQL Query

In which order are the parts of the query executed?

- **SELECT**
- **FROM**
- **WHERE**
- **GROUP BY**
- **HAVING**
- **ORDER BY**
- **LIMIT**



# Anatomy of an SQL Query

SQL clauses by order of execution:

1. FROM
2. WHERE
3. SELECT
4. GROUP BY
5. HAVING
6. ORDER BY
7. LIMIT



# Anatomy of an SQL Query

What's wrong with this query?

```
SELECT
    department,
    COUNT(*) AS number_of_employees
FROM
    employees
WHERE
    number_of_employees > 10
GROUP BY
    department
```



# Anatomy of an SQL Query

What's wrong with this query?

```
SELECT
    department,
    COUNT(*) AS number_of_employees
FROM
    employees
WHERE
    number_of_employees > 10
GROUP BY
    department
```

Conditions in the **WHERE** clause cannot reference aggregates



# Anatomy of an SQL Query

Aggregate results can only be used in the **HAVING** clause:

```
SELECT
    department,
    COUNT(*) AS number_of_employees
FROM
    employees
GROUP BY
    department
HAVING
    number_of_employees > 10
```



# Anatomy of an SQL Query

What's wrong with this query?

```
SELECT
    COUNT(*) AS number_of_employees
FROM
    employees
LIMIT
    1
```





# Anatomy of an SQL Query

What's wrong with this query?

```
SELECT
    COUNT(*) AS number_of_employees
FROM
    employees
LIMIT
    1
```

**LIMIT** is applied on the aggregated results, it has no effect!

# Common Mistakes In SQL

Writing correct SQL



# Setup

- PostgreSQL
- Use online playground [DB Fiddle](#)
- Use local database ([script](#))
- Follow along with presentation

# Setup: Sales Database

```
SELECT * FROM sale
```

| id  | branch | sold_at                | customer | product   | price | discount |
|-----|--------|------------------------|----------|-----------|-------|----------|
| --- | +      | -----                  | +        | -----     | +     | -----    |
| 1   | NY     | 2020-03-31 20:15:00-07 | Bill     | Shoes     | 10000 | 1000     |
| 2   | NY     | 2020-03-31 21:00:00-07 |          | Shoes     | 5000  | 0        |
| 3   | LA     | 2020-03-31 23:15:00-07 | Lily     | Shoes     | 15000 | 0        |
| 4   | LA     | 2020-04-01 02:10:00-07 | John     | Shoes     | 5000  | 2500     |
| 5   | NY     | 2020-03-31 20:15:00-07 |          | Shirt     | 1500  | 0        |
| 6   | NY     | 2020-03-31 19:07:00-07 | John     | Shirt     | 1850  | 0        |
| 7   | LA     | 2020-03-31 02:55:00-07 | Bill     | Shirt     | 1250  | 0        |
| 8   | LA     | 2020-03-31 03:45:00-07 | Lily     | Shirt     | 1850  | 100      |
| 9   | NY     | 2020-03-31 00:45:00-07 | Lily     | Pants     | 5200  | 0        |
| 10  | LA     | 2020-03-31 03:45:00-07 | John     | Pants     | 5200  | 0        |
| 11  | LA     | 2020-04-01 00:01:00-07 | David    | Pants     | 4500  | 0        |
| 12  | LA     | 2020-04-01 23:01:00-07 |          | Hat       | 8000  | 8000     |
| 13  | LA     | 2020-04-01 23:01:00-07 | Bill     | Give Away | 0     | 0        |
| 14  | NY     | 2020-03-31 10:01:00-07 |          | Give Away | 0     | 0        |
| 15  | LA     | 2020-04-01 03:45:00-07 |          | Give Away | 0     | 0        |

## Exercise

What is the discount rate on "Shoes"?

| id | branch | sold_at                | customer | product | price | discount |
|----|--------|------------------------|----------|---------|-------|----------|
| 1  | NY     | 2020-03-31 20:15:00-07 | Bill     | Shoes   | 10000 | 1000     |

$$\text{discount\_rate} = \text{discount} / \text{price} * 100$$

$$\text{discount\_rate} = 10.00 / 100.00 * 100 = 10\%$$



5 minutes

# Exercise

What is the discount rate on "Shoes"?

```
SELECT price, discount, discount / price * 100 as discount_rate
FROM sale
WHERE product = 'Shoes';
```

| price | discount | discount_rate |
|-------|----------|---------------|
| 10000 | 1000     | 0             |
| 5000  | 0        | 0             |
| 15000 | 0        | 0             |
| 5000  | 2500     | 0             |

😲 0? 🤔 How is this possible?



# Be Careful When Dividing Integers

Integer division truncates the result!

```
SELECT 1000 / 10000;
```

```
?column?
```

```
-----
```

```
0
```

# Be Careful When Dividing Integers

Casting the denominator to float produces the expected result

```
SELECT 1000 / 10000::float;  
?column?  
-----  
0.1
```

 [Mathematical Functions and Operators](#)





# Be Careful When Dividing Integers

💡 Add a decimal place to "mark" a constant as float:

```
SELECT 1000 / 10000.0  
?column?  
-----  
0.1
```



## Exercise

What is the discount rate on "Shoes"?

```
SELECT price, discount, discount / price::float * 100 as discount_rate
FROM sale
WHERE product = 'Shoes';
```

| price  | discount | discount_rate |
|--------|----------|---------------|
| -----+ | -----+   | -----         |
| 10000  | 1000     | 10            |
| 5000   | 0        | 0             |
| 15000  | 0        | 0             |
| 5000   | 2500     | 50            |



## Exercise

Find the average discount rate by product



5 minutes



# Exercise

Find the average discount rate by product

```
SELECT
    product,
    AVG(discount / price::float) * 100 AS discount_rate
FROM sale
GROUP BY product;
```

```
ERROR: division by zero
```



# Exercise

Why did it fail?

```
SELECT id, product, price, discount
FROM sale
WHERE price = 0;
```

| id | product   | price | discount |
|----|-----------|-------|----------|
| 13 | Give Away | 0     | 0        |
| 14 | Give Away | 0     | 0        |
| 15 | Give Away | 0     | 0        |

"Give Away" price is zero and it causes the division to fail



# Guard Against "division by zero" Errors

```
NULLIF(value1, value2)
```

Returns **NULL** if value1 equals value2, otherwise returns value1.

 [NULLIF Documentation](#)

# Guard Against "division by zero" Errors

Using `NULLIF`:

```
SELECT
    id, product, price, discount,
    NULLIF(price, 0) AS adjusted_price
FROM sale
WHERE price = 0;
```

| id | product   | price | discount | adjusted_price |
|----|-----------|-------|----------|----------------|
| 13 | Give Away | 0     | 0        |                |
| 14 | Give Away | 0     | 0        |                |
| 15 | Give Away | 0     | 0        |                |

The `adjusted_price` is `NULL` when the price equals zero

# Guard Against "division by zero" Errors

Dividing by **NULL** produces **NULL**, not an error!

```
SELECT
  id,
  product,
  discount / NULLIF(price, 0) AS discount_rate
FROM sale
WHERE price = 0;
```

| id | product   | discount_rate |
|----|-----------|---------------|
| 13 | Give Away |               |
| 14 | Give Away |               |
| 15 | Give Away |               |

Result of division is now **NULL**



# Exercise

Find the average discount rate by product

```
SELECT
    product,
    AVG(discount / NULLIF(price, 0)::float) * 100 AS discount_rate
FROM sale
GROUP BY product;
```

| product   | discount_rate      |
|-----------|--------------------|
| Shirt     | 1.3513513513513513 |
| Pants     | 0                  |
| Hat       | 100                |
| Shoes     | 15                 |
| Give Away |                    |



What is the real discount rate on "Give Away"?



# Exercise

```
COALESCE(value1, ..., valueN)
```

Returns the first of its arguments that is not **NULL**

 [COALESCE Documentation](#)

# Exercise

Find the average discount rate by product:

```
SELECT
    product,
    COALESCE(AVG(discount / NULLIF(price, 0)::float), 0) * 100
FROM sale
GROUP BY product;
```

| product   | discount_rate      |
|-----------|--------------------|
| Shirt     | 1.3513513513513513 |
| Pants     | 0                  |
| Hat       | 100                |
| Shoes     | 15                 |
| Give Away | 0                  |

Make discount rate zero for products with price zero



## Exercise

How many unique users purchased each product?



5 minutes

# Exercise

How many unique users purchased each product?

```
SELECT product, COUNT(DISTINCT customer) AS customers
FROM sale
GROUP BY product;
```

| product   | customers |
|-----------|-----------|
| Give Away | 1         |
| Hat       | 0         |
| Pants     | 3         |
| Shirt     | 3         |
| Shoes     | 3         |

🤔 Is this correct?



# Exercise

Why was this customer not counted?

```
SELECT id, produce, customer
FROM sale
WHERE product = 'Hat';
```

| id | product | customer |
|----|---------|----------|
| 12 | Hat     |          |

# Be Careful Aggregating Nullable Columns

Aggregate functions ignore **NULL** values!

```
SELECT
  product,
  COUNT(*) AS cnt,
  COUNT(customer) AS cnt_customer
FROM sale
GROUP BY product;
```

| product   | cnt | cnt_customer |
|-----------|-----|--------------|
| Shirt     | 4   | 3            |
| Pants     | 3   | 3            |
| Hat       | 1   | 0            |
| Shoes     | 4   | 3            |
| Give Away | 3   | 1            |



This can be useful...



## Exercise

How many sales were made to *known* and *unknown* customers for each product?

💡 Use the fact that aggregate functions ignore **NULL**!



# Exercise

How many sales were made to *known* and *unknown* customers for each product?

```
SELECT
    product,
    COUNT(customer) as known_customers,
    COUNT(*) - COUNT(customer) as unknown_customers
FROM sale
GROUP BY product;
```

| product   | known_customers | unknown_customers |
|-----------|-----------------|-------------------|
| Shirt     | 3               | 1                 |
| Pants     | 3               | 0                 |
| Hat       | 0               | 1                 |
| Shoes     | 3               | 1                 |
| Give Away | 1               | 2                 |

# Exercise

Write a query to find the sales made by *Bill*

```
SELECT * FROM sale WHERE customer = 'Bill';
```

| id | branch | sold_at                | customer | product   | price | discount |
|----|--------|------------------------|----------|-----------|-------|----------|
| 1  | NY     | 2020-03-31 20:15:00-07 | Bill     | Shoes     | 10000 | 1000     |
| 7  | LA     | 2020-03-31 02:55:00-07 | Bill     | Shirt     | 1250  | 0        |
| 13 | LA     | 2020-04-01 23:01:00-07 | Bill     | Give Away | 0     | 0        |

# Exercise

Write a query to find the sales made by *unknown customers*

```
SELECT * FROM sale WHERE customer IS NULL;
```

| id | branch | sold_at                | customer | product   | price | discount |
|----|--------|------------------------|----------|-----------|-------|----------|
| 2  | NY     | 2020-03-31 21:00:00-07 |          | Shoes     | 5000  | 0        |
| 5  | NY     | 2020-03-31 20:15:00-07 |          | Shirt     | 1500  | 0        |
| 12 | LA     | 2020-04-01 23:01:00-07 |          | Hat       | 8000  | 8000     |
| 14 | NY     | 2020-03-31 10:01:00-07 |          | Give Away | 0     | 0        |
| 15 | LA     | 2020-04-01 03:45:00-07 |          | Give Away | 0     | 0        |



## Exercise

Write a query to find the sales made by a customer, *using a parameter*

```
SELECT * FROM sale WHERE customer = :name;
```



Common in reporting tools

# Exercise

Write a query to find the sales made by *Bill*, using a *parameter*

```
\set name '\''Bill'\''
```

```
SELECT * FROM sale WHERE customer = :name;
```

| id | branch | sold_at                | customer | product   | price | discount |
|----|--------|------------------------|----------|-----------|-------|----------|
| 1  | NY     | 2020-03-31 20:15:00-07 | Bill     | Shoes     | 10000 | 1000     |
| 7  | LA     | 2020-03-31 02:55:00-07 | Bill     | Shirt     | 1250  | 0        |
| 13 | LA     | 2020-04-01 23:01:00-07 | Bill     | Give Away | 0     | 0        |



`\set` is a psql command to set parameters



[psql Documentation](#)

# Exercise

Write a query to find the sales made by an *unknown customer*, using a parameter

```
\set name null
```

```
SELECT * FROM sale WHERE customer = :name;
```

```
id  | branch | sold_at | customer | product | price | discount  
----+-----+-----+-----+-----+-----+-----
```

😱 No results! 🤔 Is this correct?



# Comparing NULL values

To compare **NULL** values we must use **IS**

```
SELECT NULL IS NULL;
```

```
?column?
```

```
-----
```

```
t
```

```
SELECT NULL = NULL;
```

```
?column?
```

```
-----
```

```
SELECT (NULL = NULL) IS NULL;
```

```
?column?
```

```
-----
```

```
t
```



# Comparing NULL values

Compare both **NULL** and literal values

```
SELECT *  
FROM sale  
WHERE  
    -- Handle NULL  
    (:name IS NULL AND customer IS NULL)  
OR  
    -- Handle Literal  
    (:name IS NOT NULL AND customer = :name);
```



# Exercise

Find sales made by *Bill*, using a parameter

```
\set name '\''Bill'\''
```

```
SELECT * FROM sale
WHERE (:name IS NULL AND customer IS NULL) OR (:name IS NOT NULL AND customer = :name);
```

| id | branch | sold_at                | customer | product   | price | discount |
|----|--------|------------------------|----------|-----------|-------|----------|
| 1  | NY     | 2020-03-31 20:15:00-07 | Bill     | Shoes     | 10000 | 1000     |
| 7  | LA     | 2020-03-31 02:55:00-07 | Bill     | Shirt     | 1250  | 0        |
| 13 | LA     | 2020-04-01 23:01:00-07 | Bill     | Give Away | 0     | 0        |

This works!

# Exercise

Find sales made by an *unknown customers*, using a *parameter*

```
\set name null
```

```
SELECT * FROM sale
WHERE (:name IS NULL AND customer IS NULL) OR (:name IS NOT NULL AND customer = :name);
```

| id | branch | sold_at                | customer | product   | price | discount |
|----|--------|------------------------|----------|-----------|-------|----------|
| 2  | NY     | 2020-03-31 21:00:00-07 |          | Shoes     | 5000  | 0        |
| 5  | NY     | 2020-03-31 20:15:00-07 |          | Shirt     | 1500  | 0        |
| 12 | LA     | 2020-04-01 23:01:00-07 |          | Hat       | 8000  | 8000     |
| 14 | NY     | 2020-03-31 10:01:00-07 |          | Give Away | 0     | 0        |
| 15 | LA     | 2020-04-01 03:45:00-07 |          | Give Away | 0     | 0        |

This also works



# Comparing NULL values

The complicated query works, but there must be a better way!

 **There is!**



# Comparing NULL values

```
a IS DISTINCT FROM b  
a IS NOT DISTINCT FROM b
```

Treat **NULL** like an ordinary value

 [Comparison Functions and Operators](#)

 [The Many Faces of DISTINCT in PostgreSQL](#)



## Exercise

Write a query to find the sales made by a customer, *using a parameter*

```
SELECT *  
FROM sale  
WHERE customer IS NOT DISTINCT FROM :name;
```

# Exercise

Find sales made by *Bill*, using a parameter

```
\set name '\''Bill'\''
```

```
SELECT * FROM sale WHERE customer IS NOT DISTINCT FROM :name;
```

| id | branch | sold_at                | customer | product   | price | discount |
|----|--------|------------------------|----------|-----------|-------|----------|
| 1  | NY     | 2020-03-31 20:15:00-07 | Bill     | Shoes     | 10000 | 1000     |
| 7  | LA     | 2020-03-31 02:55:00-07 | Bill     | Shirt     | 1250  | 0        |
| 13 | LA     | 2020-04-01 23:01:00-07 | Bill     | Give Away | 0     | 0        |

This works!

# Exercise

Find sales made to an *unknown customer*, using a *parameter*

```
\set name null
```

```
SELECT * FROM sale WHERE customer IS NOT DISTINCT FROM :name;
```

| id | branch | sold_at                | customer | product   | price | discount |
|----|--------|------------------------|----------|-----------|-------|----------|
| 2  | NY     | 2020-03-31 21:00:00-07 |          | Shoes     | 5000  | 0        |
| 5  | NY     | 2020-03-31 20:15:00-07 |          | Shirt     | 1500  | 0        |
| 12 | LA     | 2020-04-01 23:01:00-07 |          | Hat       | 8000  | 8000     |
| 14 | NY     | 2020-03-31 10:01:00-07 |          | Give Away | 0     | 0        |
| 15 | LA     | 2020-04-01 03:45:00-07 |          | Give Away | 0     | 0        |

# IS DISTINCT FROM

```
SELECT
  a, b,
  a = b as equal,
  a IS NOT DISTINCT FROM b AS is_not_distinct_from
FROM (VALUES
  (1, 1), (1, 2), (1, NULL), (NULL, NULL)
) AS t(a, b);
```

| a | b | equal | is_not_distinct_from |
|---|---|-------|----------------------|
| 1 | 1 | t     | t                    |
| 1 | 2 | f     | f                    |
| 1 |   |       | f                    |
|   |   |       | t                    |





# Comparing NULL values

## Database support

- PostgreSQL, SQLite: `IS DISTINCT FROM`
- MySQL: `<=>`
- SQLServer:  (deprecated `ANSI_NULLS` )
- Oracle:  (`NVL2`  can come in handy)



# Summary

- **Be careful dividing integers**

Dividing an integer by an integer truncates the result

- **Guard against "division by zero" errors**

Use `NULLIF` and `COALESCE`

- **Be careful aggregating `NULL` values**

Aggregate functions ignore `NULL` values

- **Be careful comparing `NULL` values**

Use `IS DISTINCT FROM` to treat `NULL` like a value

Mistakes you can now avoid!

# Common Mistakes In SQL

Working with dates and times

# Exercise

Find the total sold amount (excluding discounts) during each month

```
month          | total_sales
-----+-----
2020-03-01    | ?$
2020-04-01    | ?$
```

 5 minutes

 Use `date_trunc` to get the month



# Working with dates and times

How is date represented in the database?

- **EPOCH:** 01/01/1970 00:00:00 UTC
- **Unix time:** Seconds since EPOCH

```
SELECT extract('epoch' FROM now());
```

```
date_part
```

```
-----
```

```
1583585129.063067
```



# Working with dates and times

The year 2038 problem (Y2k38)

```
SET TIME ZONE UTC;  
SELECT '1970-01-01 UTC'::timestamptz + interval '1 second' * (pow(2, 31) - 1) AS y28k;
```

y28k

-----  
2038-01-19 03:14:07+00

On 32-bit systems, signed integer can only go as far as  
2038-01-19 03:14:07 UTC

# Working with dates and times

- **UTC, Coordinated Universal Time:** the primary time standards, not adjusted to daylight savings
- **GMT, Greenwich Mean Time:** synonym for UTC (time at Greenwich, England)

```
SELECT  
  now() at time zone 'UTC' as utc,  
  now() at time zone 'GMT' as gmt;
```

```
utc | 2022-11-12 14:07:03.139617
```

```
gmt | 2022-11-12 14:07:03.139617
```

 [Timezones: Documentation](#)



# Timezone

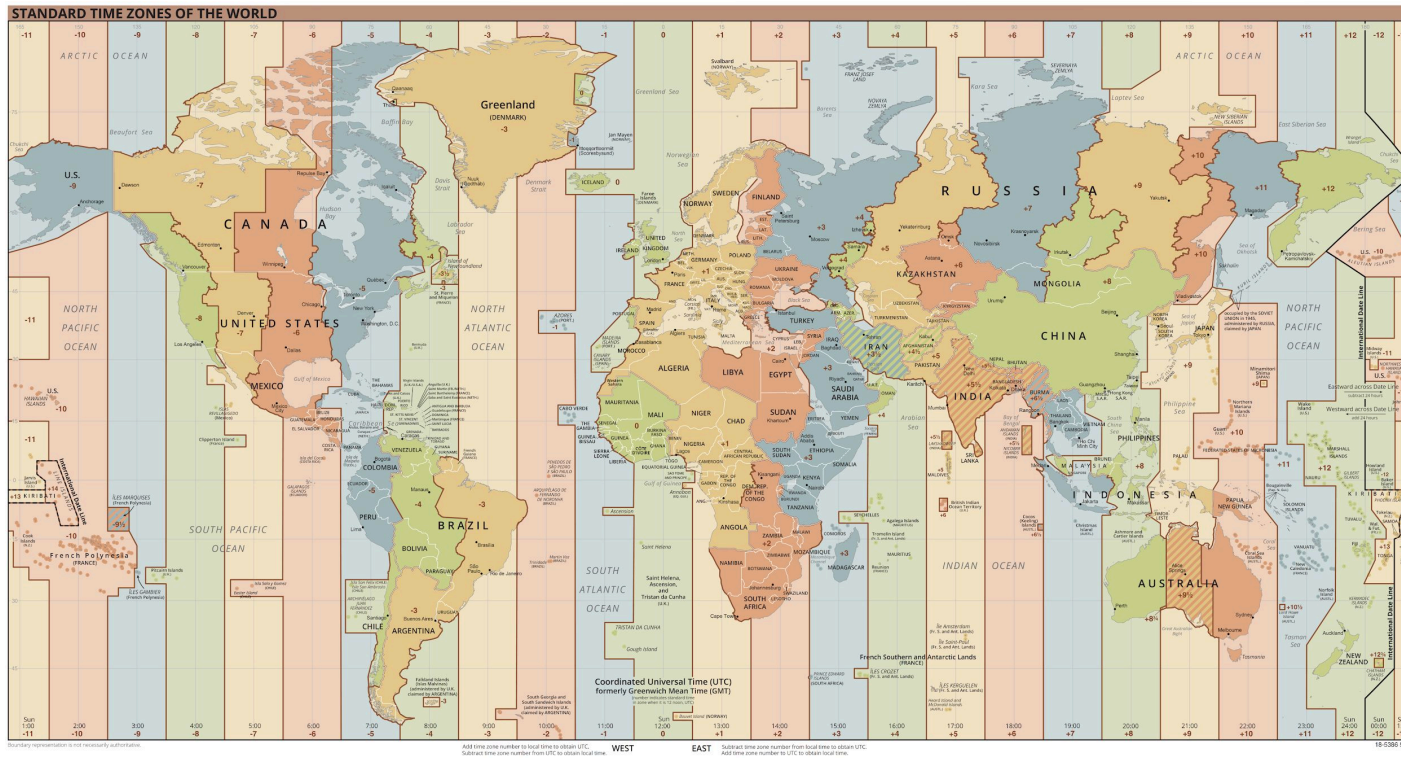
Represented as an offset from UTC

```
SELECT *  
FROM pg_timezone_names  
WHERE name ~* 'Sydney|Tel_Aviv|Paris|London|New_York'  
AND name NOT LIKE 'posix%'  
ORDER BY utc_offset;
```

| name                    | abbrev | utc_offset | is_dst |
|-------------------------|--------|------------|--------|
| -----+-----+-----+----- |        |            |        |
| America/New_York        | EST    | -05:00:00  | f      |
| Europe/London           | GMT    | 00:00:00   | f      |
| Europe/Paris            | CET    | 01:00:00   | f      |
| Asia/Tel_Aviv           | IST    | 02:00:00   | f      |
| Australia/Sydney        | AEDT   | 11:00:00   | t      |



# Timezone



[IANA: Database of all the time zones](#)



[Image credit](#)



# Working with dates and times

Specify the time zone of a timestamp

```
SELECT '2020-03-22 17:00:00 America/New_York'::timestampz;
```



# Working with dates and times

Convert a timestamp to a different time zone

```
SELECT '2020-03-22 17:00:00 America/New_York'::timestampz AT TIME ZONE 'Australia/Sydney';  
timezone
```

```
-----  
2020-03-23 08:00:00
```

```
SELECT now() AT TIME ZONE 'Australia/Sydney'  
timezone
```

```
-----  
2020-03-08 00:13:37.826917
```



## Exercise

What is the time in *your* time zone?



1 minute



Query `pg_timezone_names` to find the name of your time zone



Use `SELECT NOW() at time zone '<NAME OF YOUR TIME ZONE>'`



# Daylight Saving

Changes to the clock to extend daylight in warm seasons



# Daylight Saving

In the US the clock moves on the second Sunday in March (13/3/2022):

```
-- "Standard Time"
SELECT '2022-03-01 12:00:00 America/New_York' AT TIME ZONE 'UTC';
timezone
-----
2022-03-01 17:00:00

-- "Daylight Time" (AKA "Summer Time"):
SELECT '2022-03-13 12:00:00 America/New_York' AT TIME ZONE 'UTC';
timezone
-----
2022-03-13 16:00:00
```



# Daylight Saving

Can produce surprising results:

```
SELECT
  '2022-03-12 12:00:00 America/New_York'::timestampz
- '2022-03-13 12:00:00 America/New_York'::timestampz as diff;

diff
-----
-23:00:00
```

The day the clock moves there's only 23 hours!



# Working with dates and times

## Database types

- `date`: date without time
- `timestamp`: date and time without time zone
- `timestamp with time zone` / `timestampz`: date and time with time zone
- `interval`: period of time, useful for date arithmetics

 [Date/Time Types Documentation](#)





# Working with dates and times

## Database types

```
\d sale
```

```
Table "public.sale"
```

```
Column      | Type
```

```
-----+-----
```

```
id           | integer
```

```
branch       | text
```

```
sold_at      | timestamp with time zone
```

```
customer     | text
```

```
product      | text
```

```
price        | integer
```

```
discount     | integer
```

# Exercise: What Went Wrong...

Find the amount of sales during each month

```
SET TIME ZONE 'America/New_York';

SELECT
    date_trunc('month', sold_at) AS month,
    SUM(price) AS total_sales
FROM sale GROUP BY month;
```

| month                  | total_sales |
|------------------------|-------------|
| 2020-04-01 00:00:00-04 | 37500       |
| 2020-03-01 00:00:00-05 | 26850       |

```
SET TIME ZONE 'America/Los_Angeles';

SELECT
    date_trunc('month', sold_at) AS month,
    SUM(price) AS total_sales
FROM sale GROUP BY month;
```

| month                  | total_sales |
|------------------------|-------------|
| 2020-04-01 00:00:00-07 | 17500       |
| 2020-03-01 00:00:00-08 | 46850       |



# Working with dates and times

Unless explicitly mentioned, timezone is usually set by the client application

```
show time zone;
```

```
TimeZone
```

```
-----
```

```
America/New_York
```



# Exercise

Timezone is explicitly set to "America/New\_York":

```
SELECT
    date_trunc('month', sold_at at time zone 'America/New_York') AS month,
    SUM(price) AS total_sales
FROM sale
GROUP BY month;
```

| month               | total_sales |
|---------------------|-------------|
| -----+-----         |             |
| 2020-04-01 00:00:00 | 37500       |
| 2020-03-01 00:00:00 | 26850       |



## Exercise

What is the busiest *hour of the day* in all branches?

-  5 minutes
-  Which time zone should you use to extract the hour?

# Exercise

What is the busiest *hour of the day* in all branches?

```
SELECT extract('hour' FROM sold_at) AS hour_of_day, COUNT(*) AS sales
FROM sale
GROUP BY hour_of_day
ORDER BY sales desc;
```

| hour_of_day | sales |
|-------------|-------|
| 10          | 3     |
| 6           | 3     |
| 9           | 2     |
| 3           | 2     |
| ...         |       |

🤔 Is this correct?

# Exercise

What is the busiest *hour of the day* in all branches?

```
SELECT extract('hour' FROM sold_at AT TIME ZONE
CASE branch
    WHEN 'NY' then 'America/New_York'
    WHEN 'LA' then 'America/Los_Angeles'
END) AS hour_of_day,
COUNT(*) AS sales
FROM sale
GROUP BY hour_of_day
ORDER BY sales DESC;
```

| hour_of_day | sales |
|-------------|-------|
| 23          | 5     |
| 3           | 4     |
| 0           | 2     |
| 2           | 2     |
| 22          | 1     |



# Working with dates and times

Always be explicit about the time zone!

 Wrong!

```
date_part('month', sold_at)
```

 Right!

```
date_part('month', sold_at at time zone 'America/New_York')
```





# Working with dates and times

Always be explicit about the time zone!

 Wrong!

```
extract('month' from sold_at)
```

 Right!

```
extract('month' from sold_at at time zone 'America/New_York')
```



# Working with dates and times

Always be explicit about the time zone!

 Wrong!

```
sold_at::date
```

 Right!

```
(sold_at at time zone 'America/New_York')::date
```



# Working with dates and times

Always be explicit about the time zone!

 Wrong!

```
'2020-03-22 11:00'
```

 Right!

```
'2020-03-22 11:00 America/New_York'
```



## Exercise

How many sales were there in *March*?

In “America/New\_York” time



3 minutes



# Exercise

How many sales were there in *March*?

```
SELECT count(*)  
FROM sale  
WHERE sold_at BETWEEN '2020-03-01 America/New_York' AND '2020-04-01 America/New_York';
```

```
count  
-----  
9
```



# Exercise

How many sales were there in *April*?

```
SELECT count(*)  
FROM sale  
WHERE sold_at BETWEEN '2020-04-01 America/New_York' AND '2020-05-01 America/New_York';
```

```
count  
-----  
7
```

# Exercise

What can possibly go wrong?

```
SELECT count(*) FROM sale;
```

```
count
```

```
-----
```

```
15
```

March (9) + April (7) = 16 🤔



# BETWEEN is Inclusive!

One sale was counted twice

```
SELECT *  
FROM sale  
WHERE sold_at = '2020-04-01 America/New_York';
```

| id | branch | sold_at                | customer | product | price | discount |
|----|--------|------------------------|----------|---------|-------|----------|
| 2  | NY     | 2020-03-31 21:00:00-07 |          | Shoes   | 5000  | 0        |



# BETWEEN is Inclusive!

Use half open ranges (ranges that don't overlap):

```
SELECT COUNT(*) FROM sale
WHERE sold_at >= '2020-03-01 America/New_York' AND sold_at < '2020-04-01 America/New_York';
```

count

-----

8

```
SELECT COUNT(*) FROM sale
WHERE sold_at >= '2020-04-01 America/New_York' AND sold_at < '2020-05-01 America/New_York';
```

count

-----

7

March (8) + April (7) = 15 🤖



# BETWEEN is Inclusive!

- **Timestamps**

Unlikely if data is created by users (odd times)

More likely when data is created automatically (by batch jobs etc.)

- **Date**

Common when filtering on range of dates

- **Integers**

Binning / bucketing, searching for ranges...

```
-- Search for 90's babies... WRONG!
```

```
SELECT * FROM birthdates WHERE birthdate BETWEEN '1990-01-01' AND '2000-01-01';
```



# Summary

- **Working with date and times**
  - How is date represented in the database
  - How and when to use time zones
  - Database types: `date`, `timestamp`, `timestampz`, `interval`
  - Useful functions: `now`, `date_trunc`...
- **Be explicit about the time zone when working with times**
  - Otherwise you might get incorrect results!
- **BETWEEN is inclusive**
  - Use half open ranges to avoid overlap

# Writing Faster SQL

Evaluate and Optimize SQL Performance



# The Path of a Query

## 1. Connection

Client application establish a connection to the database server

## 2. Parser Stage

Check the query for syntax errors

## 3. Rewrite System

Adjustments to query that don't affect the result (inline views etc.)

## 4. Planner / Optimizer

Evaluate the query and find the best execution plan

## 5. Executor

Execute the query according to the execution plan



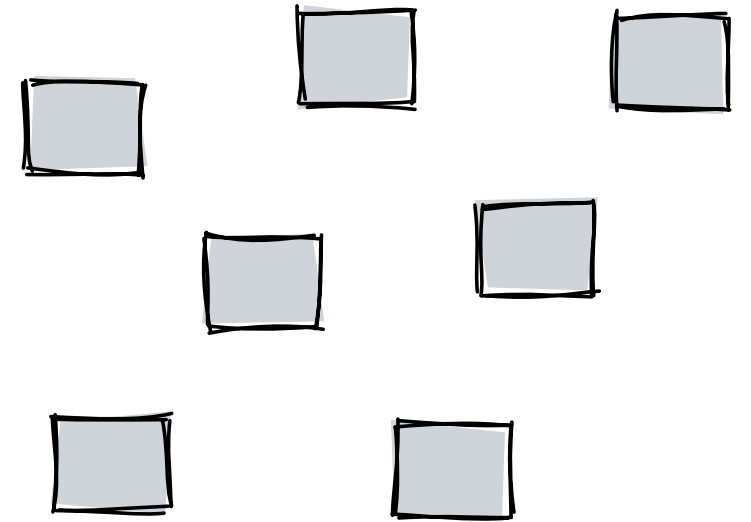
[Documentation](#)



# Query Optimizer

## 1. Generate all possible execution plans

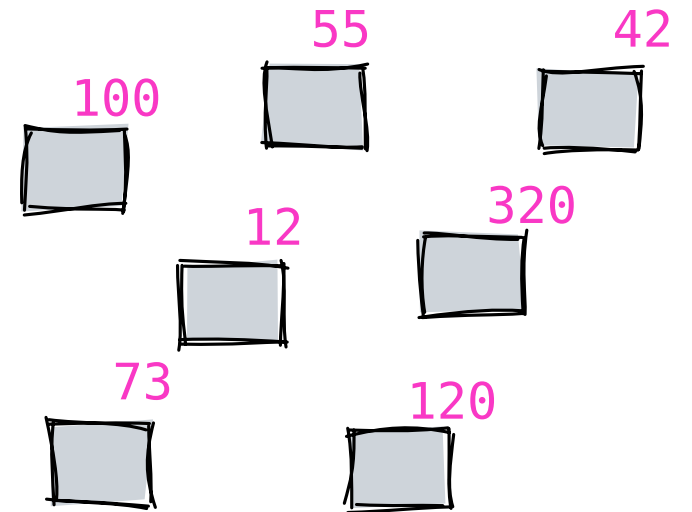
- This can take some time, depending on the query
- [Cascade of doom: Postgres update that led to 70% failure](#)



# Query Optimizer

## 2. Estimate the cost for each plan

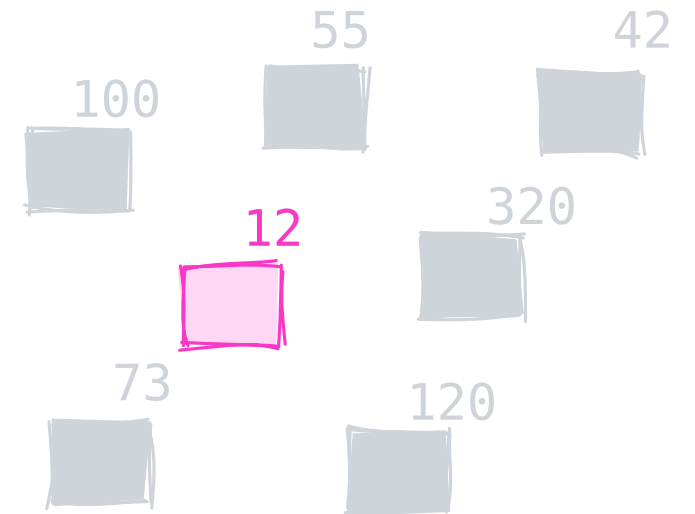
- Cost is measured in arbitrary units
- Using stats from analyzing tables and indexes
- Cost can be used to compare execution plans
- Mostly affected by IO



# Query Optimizer

## 3. Choose the plan with the lowest cost

- The lower the cost, the faster the execution is (expected) to be







# The Optimizer: What is Cost?

- Measured in arbitrary units
- Useful for comparing execution plans
- Affected by CPU, IO and other resources
- Highly affected by IO (disk access)



# The Optimizer: Rule vs. Cost based

- **Rule based optimizer**

- Using rules and heuristics to produce an execution plan
- Intuitive
- Easier to implement

- **Cost based optimizer**

- Estimate cost of all possible execution plans using statistics
- Choose the execution plan with the lowest cost
- Less intuitive
- Harder to implement
- Expected to produce better plans for most queries

 [Cockroach Labs: How We Built a Cost-Based SQL Optimizer](#)

# Generate Some Data

Can't optimize with just 15 rows...

```
INSERT INTO sale (id, branch, sold_at, customer, product, price, discount)
SELECT
  (SELECT MAX(id) FROM sale) + generate_series(1, 99985) as id,
  (ARRAY['NY', 'LA'])[ceil(random() * 3)] AS branch,
  '2020-03-01 00:00:00 UTC'::timestamp + interval '1 hour' * random() * 24 * 30 * 6 AS sold_at,
  (ARRAY['Bill', 'David', 'John', 'Lily'])[ceil(random() * 30)] AS customer,
  (ARRAY['Shoes', 'Shirt', 'Pants', 'Hat', 'Give Away'])[ceil(random() * 4)] AS product,
  round(random() * 150 * 100)::integer / 10 * 10 as price,
  0 as discount;

ANALYZE sale;
```



[Practical SQL for Data Analysis: generating data](#)



# Create Some Indexes

```
CREATE INDEX sale_customer_ix ON sale(customer);  
CREATE INDEX sale_sold_at_ix ON sale(sold_at);  
CREATE INDEX sale_product_ix ON sale(product);
```

 [PostgreSQL Indexes: Documentation](#)

 [Indexes in PostgreSQL](#)

 [Indexes in PostgreSQL: B-Tree](#)



# Create Some Indexes

```
\d sale
```

| Column   | Type                     | Nullable |
|----------|--------------------------|----------|
| id       | integer                  | not null |
| branch   | text                     |          |
| sold_at  | timestamp with time zone |          |
| customer | text                     |          |
| product  | text                     |          |
| price    | integer                  |          |
| discount | integer                  |          |

Indexes:

```
"sale_pkey" PRIMARY KEY, btree (id)
```

```
"sale_customer_ix" btree (customer)
```

```
"sale_sold_at_ix" btree (sold_at)
```

```
"sale_product_ix" btree (product)
```

# Execution Plan: The **EXPLAIN** Command

```
EXPLAIN SELECT * FROM sale;
```

```
QUERY PLAN
```

```
-----  
Seq Scan on sale  
    (cost=0.00..1751.00 rows=100000 width=33)
```

- View execution plan
- Will not execute the query
- Shows estimates

 [Using EXPLAIN](#)



# Execution Plan: A Closer Look

```
EXPLAIN SELECT * FROM sale;  
          QUERY PLAN
```

---

```
Seq Scan on sale  
  (cost=0.00..1751.00 rows=100000 width=33)
```

- **Seq Scan on sale**: Database plans to scan the entire table
- **cost=...1751**: Estimated cost for this node
- **rows=100000**: Estimated number of output rows for this node
- **width=33**: Estimated average width of rows output for this node (in bytes)

# Table Statistics

Database is using statistics to estimate cost and produce execution plans

```
SELECT relname, relpages, reltuples
FROM pg_class
WHERE relname = 'sale';
```

| relname | relpages | reltuples |
|---------|----------|-----------|
| sale    | 751      | 100000    |

```
SHOW block_size;
8192
```

```
SELECT pg_size_pretty(pg_relation_size('sale'));
6008 kB
```

- **reltuples**: Number of rows in the table
- **relpage**: Number of blocks in the table

 [pg\\_class](#)





# How is Cost Calculated?

$$\text{cost} = \text{relpages} * \text{seq\_page\_cost} + \text{reltuples} * \text{cpu\_tuple\_cost}$$

```
SELECT relpages, reltuples FROM pg_class WHERE relname = 'sale';
```

```
relpages | 751
```

```
reltuples | 100000
```

```
SHOW seq_page_cost;
```

```
seq_page_cost | 1
```

```
SHOW cpu_tuple_cost;
```

```
cpu_tuple_cost | 0.01
```

```
EXPLAIN SELECT * FROM sale;
```

```
QUERY PLAN
```

```
-----  
Seq Scan on sale (cost=0.00..1751.00 rows=100000 width=33)
```

$$1,751 = 751 * 1 + 100,000 * 0.01$$

# Column Statistics

```
SELECT attname, null_frac, n_distinct, correlation
FROM pg_stats WHERE tablename = 'sale';
```

| attname  | null_frac | n_distinct | correlation  |
|----------|-----------|------------|--------------|
| sold_at  | 0         | -1         | -0.004219238 |
| customer | 0.8694    | 4          | 0.2778079    |
| product  | 0         | 5          | 0.2626033    |
| price    | 0         | 1501       | 0.0019420487 |
| discount | 0         | 1          | 1            |
| branch   | 0         | 2          | 0.4909097    |
| id       | 0         | -1         | 1            |



# Column Statistics

**null\_frac**: % null values

| attname  | null_frac | n_distinct | correlation  |
|----------|-----------|------------|--------------|
| sold_at  | 0         | -1         | -0.004219238 |
| customer | 0.8694    | 4          | 0.2778079    |
| product  | 0         | 5          | 0.2626033    |
| price    | 0         | 1501       | 0.0019420487 |
| discount | 0         | 1          | 1            |
| branch   | 0         | 2          | 0.4909097    |
| id       | 0         | -1         | 1            |

- In 86.94% percent of rows the column **customer** is **NULL**



# Column Statistics

**n\_distinct** (positive): Number of distinct values

| attname  | null_frac | n_distinct | correlation  |
|----------|-----------|------------|--------------|
| sold_at  | 0         | -1         | -0.004219238 |
| customer | 0.8694    | 4          | 0.2778079    |
| product  | 0         | 5          | 0.2626033    |
| price    | 0         | 1501       | 0.0019420487 |
| discount | 0         | 1          | 1            |
| branch   | 0         | 2          | 0.4909097    |
| id       | 0         | -1         | 1            |

- **customer** has 4 distinct values
- **product** has 5 distinct values
- **branch** has 2 distinct values



# Column Statistics

**n\_distinct** (negative):  $-1 * \text{distinct\_values} / \text{rows}$

| attname  | null_frac | n_distinct | correlation  |
|----------|-----------|------------|--------------|
| sold_at  | 0         | -1         | -0.004219238 |
| customer | 0.8694    | 4          | 0.2778079    |
| product  | 0         | 5          | 0.2626033    |
| price    | 0         | 1501       | 0.0019420487 |
| discount | 0         | 1          | 1            |
| branch   | 0         | 2          | 0.4909097    |
| id       | 0         | -1         | 1            |

- Number of distinct values is likely to increase as table grows
- **-1** means unique

# Column Statistics

**correlation**: Correlation between physical row ordering and logical ordering of the column values

| attname  | null_frac | n_distinct | correlation  |
|----------|-----------|------------|--------------|
| sold_at  | 0         | -1         | -0.004219238 |
| customer | 0.8694    | 4          | 0.2778079    |
| product  | 0         | 5          | 0.2626033    |
| price    | 0         | 1501       | 0.0019420487 |
| discount | 0         | 1          | 1            |
| branch   | 0         | 2          | 0.4909097    |
| id       | 0         | -1         | 1            |

- **1** means rows stored sorted in ascending order on disk
- Affect index performance



# Execution Plan

```
EXPLAIN SELECT * FROM sale WHERE id = 1000;
```

```
QUERY PLAN
```

```
-----  
Index Scan using sale_pkey on sale (cost=0.29..8.31 rows=1 width=33)  
  Index Cond: (id = 1000)
```

- Database used the index on the primary key (`sale_pkey`)
- Estimated cost is 8
- Database estimates 1 row in result



# Execution Plan

```
EXPLAIN SELECT * FROM sale
WHERE customer IS NULL;
```

## QUERY PLAN

```
-----
Seq Scan on sale (cost=0.00..1751.00 rows=86940 width=33)
  Filter: (customer IS NULL)
```

$$rows = reltuples * null\_frac$$

$$86,940 = 100,000 * 0.8694$$





# Execution Plan

```
EXPLAIN SELECT * FROM sale
WHERE customer IS NOT NULL;
```

## QUERY PLAN

---

```
Seq Scan on sale (cost=0.00..1751.00 rows=13060 width=33)
  Filter: (customer IS NOT NULL)
```

$$rows = reltuples * (1 - null\_frac)$$

$$13,060 = 100,000 * (1 - 0.8694)$$

# Column Statistics

Common values and their corresponding frequency

```
SELECT attname, most_common_vals, most_common_freqs
FROM pg_stats
WHERE tablename = 'sale'
AND attname IN ('branch', 'product', 'customer');
```

| attname  | most_common_vals           | most_common_freqs                  |
|----------|----------------------------|------------------------------------|
| customer | {Lily, David, Bill, John}  | {0.03516, 0.0319, 0.0317, 0.03176} |
| product  | {Shoes, Pants, Hat, Shirt} | {0.25396, 0.2505, 0.2491, 0.24636} |
| branch   | {NY, LA}                   | {0.50433, 0.4956}                  |

# Cardinality Estimate

## Sales in NY

```
EXPLAIN SELECT * FROM sale
WHERE branch = 'NY';
```

### QUERY PLAN

```
-----
Seq Scan on sale (cost=0.00..2001.00 rows=50433 width=33)
  Filter: (branch = 'NY'::text)
```

$$rows = reltuples * NY\ freq$$

$$50,433 = 100,000 * 0.50433$$

# Cardinality Estimate

Sales in LA

```
EXPLAIN SELECT * FROM sale
WHERE branch = 'LA';
```

QUERY PLAN

```
-----
Seq Scan on sale (cost=0.00..2001.00 rows=49567 width=33)
  Filter: (branch = 'LA'::text)
```

$$rows = reltuples * LAfreq$$

$$49,567 = 100,000 * 0.49567$$



# Cardinality Estimate

Sales in a city that **does not exist in table**

```
EXPLAIN SELECT * FROM sale
WHERE branch = 'F00';
```

## QUERY PLAN

```
-----
Seq Scan on sale (cost=0.00..2001.00 rows=1 width=33)
  Filter: (branch = 'F00'::text)
```

For internal reasons the row estimate cannot be zero, so 1 is used instead

# Cardinality Estimate

## Sales of Shoes and Pants

```
EXPLAIN SELECT * FROM sale
WHERE product IN ('Shoes', 'Pants');
```

### QUERY PLAN

```
-----
Seq Scan on sale (cost=0.00..2001.00 rows=50447 width=33)
  Filter: (product = ANY ('{Shoes,Pants}'::text[]))
```

$$rows = reltuples * (Shoes\ freq + Pants\ freq)$$

$$50,447 = 100,000 * (0.253966666 + 0.2505)$$

# Cardinality Estimate

## Sales of Shoes in NY

```
EXPLAIN SELECT * FROM sale
WHERE branch = 'NY' AND product = 'Shoes';
```

### QUERY PLAN

```
-----
Seq Scan on sale (cost=0.00..2251.00 rows=12808 width=33)
  Filter: ((branch = 'NY'::text) AND (product = 'Shoes'::text))
```

$$rows = reltuples * (NY\ freq * Shoes\ freq)$$

$$12,808 = 100,000 * (0.5043333 * 0.25396666)$$

# Cardinality Estimate

## Sales of Shoes or in NY

```
EXPLAIN SELECT * FROM sale
WHERE branch = 'NY' OR product = 'Shoes';
```

### QUERY PLAN

```
-----
Seq Scan on sale (cost=0.00..2251.00 rows=63022 width=33)
  Filter: ((branch = 'NY'::text) OR (product = 'Shoes'::text))
```

$$rows = reltuples * (NYfreq + Shoesfreq - (NYfreq * Shoesfreq))$$

$$63,022 = 100,000 * (0.5043 + 0.25396 - (0.5043 * 0.25396))$$



# Estimate vs. Actual Rows

```
EXPLAIN (ANALYZE ON, TIMING ON)
```

```
SELECT * FROM sale;
```

QUERY PLAN

```
-----  
Seq Scan on sale (cost=0.00..1751 rows=100000 width=33)  
      (actual time=0.026..11.209 rows=100000 loops=1)
```

```
    Planning Time: 0.105 ms
```

```
    Execution Time: 15.568 ms
```

- Actually executes the query
- Show planning and execution time
- Show actual rows



# Cardinality Estimate

How accurate are estimates?

```
EXPLAIN (ANALYZE ON) SELECT * FROM sale WHERE branch = 'NY' OR product = 'Shoes';
                                QUERY PLAN
-----
Seq Scan on sale (cost=0.00..2251.00 rows=62683 width=33) (actual time=0.026..36.181 rows=62678 loops=1)
  Filter: ((branch = 'NY'::text) OR (product = 'Shoes'::text))
  Rows Removed by Filter: 37322
Planning Time: 0.180 ms
Execution Time: 40.728 ms
```

- Estimated -> 62,683
- Actual -> 62,678

**Pretty Accurate!**



# Identifying Bad Plans

A proposed approach to identify potential issues in execution plans using cardinality feedback:

1. Produce execution plan with actual rows (`EXPLAIN ANALYZE`)
2. Identify nodes with significant difference between estimate and actual rows
3. Investigate reason for bad estimate (stale stats, bad query, etc.)



# Statistics: Recap

- The database keeps stats on tables, indexes and columns
- Statistics are used to calculate cardinality estimates
- Cardinality estimates are used to calculate cost
- Cost is used to determine the best execution plan



# Statistics: Key Takeaway

**The optimizer is only as good as its cardinality estimates!**

👉 It is your job to make sure the database can produce accurate cardinality estimates



# Collecting Statistics

Collect statistics explicitly

```
-- Analyze all tables in database  
ANALYZE;
```

```
-- Analyze specific table  
ANALYZE sales;
```

```
-- Analyze specific columns  
ANALYZE sale(branch, customer);
```



Useful after bulk load into a table



# Collecting Statistics

Display progress data

```
db=# ANALYZE VERBOSE sale;  
INFO: analyzing "public.sale"  
INFO: "sale": scanned 751 of 751 pages, containing 100000 live rows and 0 dead rows;  
30000 rows in sample, 100000 estimated total rows  
ANALYZE
```



Useful when analyzing big tables



# Collecting Statistics

Automatically collect statistics

- "autovacuum" is a routine maintenance task performed by the database
- The database can analyze tables and indexes as they are vacuumed
- Triggered automatically after a certain amount of changes to the tables has been reached





# Collecting Statistics

Check if autovacuum is enabled

```
show autovacuum;  
autovacuum  


---

on
```



# Collecting Statistics

Check when analyzed

```
db=# SELECT last_analyze, last_autoanalyze
FROM pg_stat_all_tables
WHERE relname = 'sale';
```

```
-[ RECORD 1 ]-----+-----
last_analyze   | 2022-11-04 11:52:18.473069+02
last_autoanalyze | 2022-10-22 18:17:18.391999+03
```



# Collecting Statistics: Recap

- Optimizer uses statistics to produce execution plans
- Keeping statistics up to date has a significant effect on query performance
- Statistics can be collected explicitly using `ANALYZE`, or in the background by enabling autovacuum



# Execution Plan

From the [documentation](#)

*“ Plan-reading is an art that requires some experience to master...”*



# Execution Plan: Takeaways

- **Databases are really good at optimizing queries**
  - The optimizer is only as good as its stats
  - Incorrect estimates can indicate a possible performance problem
  - Keep fresh statistics by analyzing after bulk loads and enabling autovacuum
- **Use cost to compare execution plans**
  - The lower the cost, the faster the query is expected to be
  - Cost is estimated based on statistics, so it can be inaccurate
  - Cost is not an absolute measure, use it only for comparison



# Influencing Execution Plans

- **Benchmarking and simulation**

*How would this plan look w/o this index?*

- **Maintaining plan stability**

*How can I make sure this execution plan doesn't change?*



# Optimizer Parameters

Plan with `seq_page_cost=1` using index:

```
db=# SHOW seq_page_cost;  
seq_page_cost
```

---

1

```
db=# EXPLAIN SELECT * FROM sale WHERE id < 50000;  
QUERY PLAN
```

---

```
Index Scan using sale_pkey on sale (cost=0.29..1799.48 rows=49725 width=33)  
Index Cond: (id < 50000)
```

# Optimizer Parameters

Plan with `seq_page_cost=0.1` using full table scan:

```
db=# SET seq_page_cost to 0.1;  
SET
```

```
db=# EXPLAIN SELECT * FROM sale WHERE id < 50000;  
QUERY PLAN
```

---

```
Seq Scan on sale (cost=0.00..1325.10 rows=49725 width=33)  
  Filter: (id < 50000)
```

Reduce weight -> cost of seq scan decreases -> seq scan is selected as best plan 🏆





# Optimizer Parameters

All features enabled:

```
db=# EXPLAIN SELECT * FROM sale WHERE id = 1000;
```

```
QUERY PLAN
```

---

```
Index Scan using sale_pkey on sale (cost=0.29..8.31 rows=1 width=33)
```

```
Index Cond: (id = 1000)
```

Index scan is used.



# Optimizer Parameters

Disable index scan:

```
db=# set enable_indexscan to off;
db=# EXPLAIN SELECT * FROM sale WHERE id = 1000;

```

---

```
Bitmap Heap Scan on sale (cost=4.30..8.31 rows=1 width=33)
  Recheck Cond: (id = 1000)
    -> Bitmap Index Scan on sale_pkey (cost=0.00..4.30 rows=1 width=0)
          Index Cond: (id = 1000)
```

Bitmap index scan is used instead!



# Optimizer Parameters

Disabled bitmap index scans:

```
db=# set enable_bitmapscan to off;
db=# EXPLAIN SELECT * FROM sale WHERE id = 1000;

```

---

```
Seq Scan on sale (cost=0.00..2001.00 rows=1 width=33)
  Filter: (id = 1000)
```

Full table scan is used!



# Optimizer Parameters: Recap

- Adjust planner cost constants
- Enable / disable planner features

 [Query Planning: Server Configuration](#)



# Optimizer Hints

Provide specific instructions on how to execute a query

```
db=# /*+  
db*#      SeqScan(sale)  
db*#      */  
db=# SELECT * FROM sale WHERE id = 1000;
```

Hints override the optimizer decisions



# Optimizer Hints: Recap

## Big debate in the PostgreSQL community!

- Supported by some big engines (such as Oracle)
- Not supported in PostgreSQL core
- Can use extension [pg\\_hint\\_plan](#)

 [Optimizer hints discussion on PostgreSQL Wiki](#)



# Changing the Query

Write the query in different ways, for example:

- `JOIN` vs. `IN` vs. `EXISTS` vs. `ANY`
- `NOT IN` vs. `NOT EXISTS` vs. `LEFT JOIN` & `IS NULL`

 [SQL Optimizations in PostgreSQL: IN vs EXISTS vs ANY/ALL vs JOIN](#)



# Changing the Query: Recap

**Very limited!**

PostgreSQL is usually smart enough to treat logically equivalent queries the same





# Faux Predicate

Query sales that were modified until some date

```
db=# EXPLAIN SELECT * FROM sale WHERE modified < '2019-01-01 UTC';  
               QUERY PLAN  
  
-----  
Seq Scan on sale (cost=0.00..510225.35 rows=1357 width=276)  
  Filter: (modified < '2019-01-01 00:00:00'::timestamp with time zone)
```

No index on **modified** -> database will scan the entire table



# Faux Predicate

- 😞 There is no index on `modified`
- 🤔 A sale cannot be modified before it is created...
- 😊 There is an index on `created`

Add a predicate using `created` to the query!

# Faux Predicate

## Query with faux predicate

```
db=# EXPLAIN SELECT * FROM sale
WHERE modified < '2019-01-01 UTC'
AND created < '2019-01-01 UTC';
```

### QUERY PLAN

```
-----
Index Scan using sale_created_ix on sale (cost=0.44..4.52 rows=1 width=276)
  Index Cond: (created < '2019-01-01 00:00:00'::timestamp with time zone)
  Filter: (modified < '2019-01-01 00:00:00'::timestamp with time zone)
```

Faux predicate should not change the outcome of the query!

 [Faux Predicate](#)



## Faux Predicate: Recap

- Communicate a logical dependency that the database cannot know about any other way
- Should not change the outcome of the query

\* This is not a real term, I made it up – the term is borrowed from "Faux Weight" in CSS ("Faux" means "fake")



# Optimization Fences

Common table expressions (**WITH** clause) can be materialized and act as "optimization fences"

```
EXPLAIN SELECT * FROM sale WHERE branch = 'LA' AND customer = 'Bill';  
QUERY PLAN
```

---

```
Bitmap Heap Scan on sale (cost=37.53..838.18 rows=1652 width=33)  
  Recheck Cond: (customer = 'Bill'::text)  
  Filter: (branch = 'LA'::text)  
    -> Bitmap Index Scan on sale_customer_ix (cost=0.00..37.12 rows=3310 width=0)  
        Index Cond: (customer = 'Bill'::text)
```

Database used the index on customer



# Optimization Fences

Force DB to apply filter on branch first

```
EXPLAIN WITH la_sales AS MATERIALIZED (SELECT * FROM sale WHERE branch = 'LA')
SELECT * FROM la_sales WHERE customer = 'Bill';
QUERY PLAN
```

---

```
CTE Scan on la_sales (cost=2001.00..3123.75 rows=250 width=116)
  Filter: (customer = 'Bill'::text)
CTE la_sales
  -> Seq Scan on sale (cost=0.00..2001.00 rows=49900 width=33)
       Filter: (branch = 'LA'::text)
```

Database materialized the results of the CTE



# Optimization Fences: Recap

- Materializing results consume space – be careful!
- Materializing was the default prior to PostgreSQL 12
- Starting at PostgreSQL 12, **WITH** is not materializing unless the **MATERIALIZED** keyword is set explicitly

 [Be Careful With CTE in PostgreSQL](#)



# Extended Statistics

Collect statistics on expressions

```
db=# EXPLAIN SELECT * FROM sale WHERE lower(customer) = 'bill';
```

```
Seq Scan on sale (cost=0.00..2251.00 rows=500 width=33)  
  Filter: (lower(customer) = 'bill'::text)
```

```
db=# CREATE STATISTICS sale_lower_customer_name ON lower(customer) FROM sale;  
CREATE STATISTICS
```

```
db=# ANALYZE sale;  
ANALYZE
```

```
db=# EXPLAIN SELECT * FROM sale WHERE lower(customer) = 'bill';
```

```
Seq Scan on sale (cost=0.00..2251.00 rows=3303 width=33)  
  Filter: (lower(customer) = 'bill'::text)
```

Notice the cardinality estimate!



# Extended Statistics

## Collect statistics on expressions

```
db=# EXPLAIN SELECT date_trunc('month', sold_at) as month, count(*) FROM sale GROUP BY 1;
```

```
HashAggregate (cost=7626.00..9657.25 rows=100000 width=16)
  Group Key: date_trunc('month'::text, sold_at)
  Planned Partitions: 4
  -> Seq Scan on sale (cost=0.00..2001.00 rows=100000 width=8)
```

```
db=# CREATE STATISTICS sale_sold_at_month ON (date_trunc('month', sold_at)) FROM sale;
CREATE STATISTICS
```

```
db=# ANALYZE sale;
ANALYZE
```

```
db=# EXPLAIN SELECT date_trunc('month', sold_at) as month, count(*) FROM sale GROUP BY 1;
```

```
HashAggregate (cost=2501.00..2501.07 rows=6 width=16)
  Group Key: date_trunc('month'::text, sold_at)
  -> Seq Scan on sale (cost=0.00..2001.00 rows=100000 width=8)
```



# Extended Statistics

## Collect dependency statistics

```
db=# EXPLAIN SELECT * FROM sale WHERE branch = 'LA' AND customer = 'Bill';  
  
Bitmap Heap Scan on sale (cost=37.48..838.02 rows=1656 width=33)  
  Recheck Cond: (customer = 'Bill'::text)  
  Filter: (branch = 'LA'::text)  
    -> Bitmap Index Scan on sale_customer_ix (cost=0.00..37.06 rows=3303 width=0)  
        Index Cond: (customer = 'Bill'::text)  
  
db=# CREATE STATISTICS sale_branch_customer ON branch, customer FROM sale;  
CREATE STATISTICS  
  
db=# ANALYZE sale;  
ANALYZE  
  
db=# EXPLAIN SELECT * FROM sale WHERE branch = 'LA' AND customer = 'Bill';  
  
Bitmap Heap Scan on sale (cost=37.45..837.95 rows=1610 width=33)  
  Recheck Cond: (customer = 'Bill'::text)  
  Filter: (branch = 'LA'::text)  
    -> Bitmap Index Scan on sale_customer_ix (cost=0.00..37.04 rows=3300 width=0)  
        Index Cond: (customer = 'Bill'::text)
```

Notice the cardinality estimate!



# Invisible Indexes

Say you want to see what an execution plan looks like without a specific index, **what are your options?**



# Invisible Indexes

- Disable index via parameters -> may affect other indexes
- Use hints -> not always supported (requires an extension)
- Changing the query -> not always possible
- Optimization fences (WITH) -> not always possible
- Drop the index -> compromise performance of live system

 **Is there another option?**



# Invisible Indexes

Plan using an index on `customer`

```
db=# EXPLAIN SELECT * FROM sale WHERE customer = 'Bill';  
QUERY PLAN
```

---

```
Bitmap Heap Scan on sale (cost=43.39..838.11 rows=3497 width=33)  
  Recheck Cond: (customer = 'Bill'::text)  
    -> Bitmap Index Scan on sale_customer_ix (cost=0.00..42.52 rows=3497 width=0)  
          Index Cond: (customer = 'Bill'::text)
```

# Invisible Indexes

Simulate execution w/o the index by dropping inside a transaction and immediately rollback:

```
db=# BEGIN;
BEGIN

db=# DROP INDEX sale_customer_ix;
DROP INDEX

db=# EXPLAIN SELECT * FROM sale WHERE customer = 'Bill';
               QUERY PLAN
-----
Seq Scan on sale  (cost=0.00..2001.00 rows=3497 width=33)
  Filter: (customer = 'Bill'::text)

db=# ROLLBACK;
ROLLBACK
```



[Make Indexes "Invisible" using transactional DDL](#)



# Influencing Execution Plans: Recap

Ways to influence the execution plan

- Optimizer parameters
- Optimizer hints
- Changing the query
- Faux predicates
- Optimization fences
- Extended statistics
- Invisible indexes (using transactional DDL)



# Influencing Execution Plans: Takeaways

- The database is very good at optimizing queries
- Your goal is to communicate to the database information about your data
- Data can change – fixating a plan may cause performance issues down the road
- Make sure to only interfere with the optimizer *only* after you exhausted conventional methods





# Common Misconception

🤔 Indexes are always best!



# Indexes are not Always Best

Create a "heystack":

```
CREATE TABLE heystack (what TEXT, padding TEXT);

-- Needle x 1
INSERT INTO heystack (what, padding)
VALUES ('needle', generate_random_string(1000));

-- Hey x 99,999
INSERT INTO heystack (what, padding)
  SELECT 'hey', generate_random_string(1000)
  FROM generate_series(1, 99999);
```



# Indexes are not Always Best

Create a index for our "heystack":

```
CREATE INDEX heystack_what_ix ON heystack(what);
```



# Indexes are not Always Best

Find the "needle":

```
db=# EXPLAIN SELECT * FROM heystack WHERE what = 'needle';
                                QUERY PLAN
-----
Index Scan using heystack_what_ix on heystack (cost=0.29..4.31 rows=1 width=1008)
  Index Cond: (what = 'needle'::text)
```

Index is used!



# Indexes are not Always Best

Find some "hey":

```
db=# EXPLAIN SELECT * FROM heystack WHERE what = 'hey';  
               QUERY PLAN
```

```
-----  
Seq Scan on heystack (cost=0.00..15536.00 rows=100000 width=1008)  
  Filter: (what = 'hey'::text)
```

Index is not used!



# Indexes are not Always Best

~~Indexes are always best!~~

- When reading a large portion of the table, a full table scan might be better
- There is no magic number



**Indexes are *not* always best!**



# Summary

- **How the database processes a query**
  - Parse -> rewrite -> plan -> execute
- **How to produce and compare execution plans**
  - Using EXPLAIN
- **How row estimates are calculated**
  - For different type of predicates, using database statistics
- **When using an index is not the best plan**
  - Accessing a lot of rows using an index can be inefficient
- **How to influence execution plans**
  - For testing, benchmarking or to maintain plan stability



## Going Further...

- [PostgreSQL Essentials: Leveling Up Your Data Work](#)  
O'Reilly on-demand class with quizzes
- [PostgreSQL Bootcamp](#)  
O'Reilly live training, May 7 & 8
- [How to get Foreign Keys Horribly Wrong in Django](#)  
DjangoCon EU Dublin, April 24



# Haki Benita

-  Check out my blog [hakibenita.com](https://hakibenita.com)
-  Find me on Twitter [@be\\_haki](https://twitter.com/be_haki)
-  Find my on LinkedIn [Haki Benita](https://www.linkedin.com/company/haki-benita)
-  Subscribe to my newsletter at [hakibenita.com/subscribe](https://hakibenita.com/subscribe)
-  Send me an email [me@hakibenita.com](mailto:me@hakibenita.com)